
SAFE-VALS: Certifiably Safe and Adaptive Variance-Bounded Sparsification for Scalable Graph Attention

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 Graph Attention Networks (GATs) achieve state-of-the-art accuracy on heteroge-
2 neous, high-degree graphs, yet their quadratic edge-attention cost blocks deploy-
3 ment at hundred-million-node scale and in safety-critical domains that demand
4 worst-case guarantees. Existing locality-sensitive-hash sparsifiers accelerate at-
5 tention only in expectation, assume static node embeddings and store per-bucket
6 statistics that dominate memory on web-scale graphs. We present SAFE-VALS, a
7 Safety-Aware, Adaptive, Feature-Evolving framework that (i) meta-learns the hash
8 function through a tiny MLP co-optimised with the GNN, (ii) replaces explicit
9 histograms by a sub-megabyte Count-Min sketch, and (iii) accepts a sparse edge
10 mask only when a Bennett-style tail bound guarantees that every soft-max coeffi-
11 cient deviates from its dense value by at most ε with probability $1 - \delta$, otherwise
12 falling back to dense attention on-the-fly. Continual bucket pruning, template
13 sharing across heads and dual-phase freezing further reduce memory and compute.
14 Experiments on five public benchmarks up to the 111 M-node OGBN-papers100M,
15 a synthetic 100 M-node power-law graph and a twelve-month temporal Reddit
16 corpus demonstrate $1.3\text{-}1.9 \times$ training speed-up, up to $3.3 \times$ inference acceleration,
17 $45\text{-}68\%$ memory reduction and $\leq 0.3\%$ accuracy loss at $\varepsilon = 0.05$, with zero
18 bound violations in ten million monitored batches. Ablations confirm that adaptive
19 hashing yields $+0.5\%$ accuracy and that the safety fallback is indispensable. The
20 combination of provable worst-case guarantees and practical efficiency positions
21 SAFE-VALS as a deployable alternative to existing sparse-attention approaches.

22 1 Introduction

23 Attention-based graph neural networks have become the de-facto choice for tasks in which nodes must
24 weight their neighbours dynamically, delivering strong results in recommendation, drug discovery
25 and fraud detection. Unfortunately, computing and normalising attention scores for every edge incurs
26 $\mathcal{O}(|E| \cdot H)$ complexity, which is prohibitive once $|E|$ reaches billions. Recent work mitigates the
27 cost through sub-graph sampling ?, neighbourhood windows ? or randomised sparse computations ?,
28 yet three obstacles remain. 1) Training-time sparsifiers usually build hash buckets from the initial
29 embeddings and ignore the continual drift that occurs as parameters update, leading to progressively
30 sub-optimal edge selection. 2) Their theoretical guarantees hold only in expectation; an individual
31 soft-max coefficient may deviate arbitrarily from its dense value—unacceptable in security, finance or
32 medicine. 3) Seemingly innocuous bookkeeping such as per-bucket counts can outweigh the memory
33 saved by pruning edges once the graph contains hundreds of millions of nodes.

34 We address all three challenges simultaneously with SAFE-VALS, a Safety-Aware, Adaptive, Feature-
35 Evolving variant of VALS-GAT. SAFE-VALS equips sparse attention with provable worst-case error
36 bounds while remaining light-weight enough to run in pure Python on commodity GPUs. Its core

ideas are: (i) meta-learned hashing that tracks embedding drift, (ii) sub-megabyte sketching of bucket sizes and (iii) a Bennett-inequality tail bound coupled with automatic dense fallback that enforces safety per mini-batch.

Extensive experiments span five public datasets, a 100 M-node synthetic graph and a year-long temporal corpus, covering scale, adversarial perturbations and feature drift. Across five random seeds SAFE-VALS delivers up to $1.9 \times$ training speed-up, $3.3 \times$ inference acceleration and zero safety violations while matching dense accuracy within 0.3%.

1.1 Contributions

- **Worst-case deviation bound:** We derive the first worst-case deviation bound for sparse graph attention and embed it in an automatic fallback mechanism.
- **Meta-learned hashing:** We introduce meta-learned hashing with product-quantised distillation that adapts to embedding drift at negligible inference cost.
- **Compressed statistics and sharing:** We compress bucket statistics via a Count-Min sketch and share edge templates across heads, cutting memory by up to 68%.
- **Comprehensive validation:** We validate the approach on web-scale graphs, adversarial noise bursts and a year-long temporal corpus, providing detailed ablations.

Looking forward, SAFE-VALS could be combined with mixed-precision quantisation ? or extended to joint edge-model sparsification ?, further bridging the gap between scalability and reliability.

2 Related Work

Sparse graph attention accelerators fall into three broad categories. Sampling-based methods such as GraphSAINT ? and GST ? draw sub-graphs every mini-batch, reducing memory and offering provably unbiased training, yet their guarantees hold only in expectation. Variance-reduced samplers tighten convergence ? but still lack per-edge safety. Kernel-level approaches, for example NATTEN ? or BRSC-GAT ?, fuse CUDA kernels for sliding windows or random masks; these excel on images or dense meshes but presuppose fixed neighbourhoods and cannot react to feature drift.

Another line sparsifies the graph itself. Unified Lottery Tickets ? and Joint Edge-Model Sparse Learning ? jointly prune edges and weights, achieving strong compression but requiring iterative retraining and again offering no worst-case control. Meta-adaptive layers like GeniePath ? learn receptive fields but still process dense attention.

Cluster-GCN ? and GNNAutoScale ? tackle scalability by clustering or re-using historical embeddings, yet they still compute over every edge inside a cluster. SAFE-VALS is orthogonal and can be plugged into these frameworks.

Closest to our work, VALS-GAT hashes query-key pairs once at initialisation and reuses the buckets throughout training. SAFE-VALS extends VALS-GAT by learning the hash online, guaranteeing worst-case rather than expected deviation, compressing bucket statistics and providing automatic dense fallback—none of which are simultaneously addressed in prior art.

3 Background

3.1 Problem setting

Let $G = (V, E)$ be a directed graph with $|V| = N$ nodes, node features $X \in \mathbb{R}^{N \times d}$ and labels on a subset of nodes. A GAT layer computes an unnormalised score $\tilde{a}_{uv} = q_u^\top k_v$ per edge (u, v) , applies a soft-max over the incoming edges of v to obtain a_{uv} and aggregates messages $m_v = \sum_{u \in \mathcal{N}(v)} a_{uv} W h_u$. The cost is $\mathcal{O}(|E| \cdot H)$, quadratic in neighbourhood size.

Sparsified attention retains at most K edges per bucket B , yielding approximate weights $\hat{a}_{uv}$. Prior analyses bound the expectation, but safety-critical deployments require $\mathbb{P}(|\hat{a} - a| > \varepsilon) \leq \delta$ for all edges at all steps.

82 3.2 Count-Min sketch

83 A Count-Min sketch with width w and depth d maintains streaming counts using $\mathcal{O}(wd)$ memory
 84 independent of $|E|$, returning an unbiased estimate $\hat{c}_B$ of the true bucket cardinality $|B|$ with relative
 85 error approximately $1/w$.

86 3.3 Bennett’s inequality

87 For independent bounded variables z_i with $|z_i| \leq \alpha_{\max}$, Bennett’s inequality gives

$$\mathbb{P}\left(\left|\sum_i z_i\right| > \varepsilon\right) \leq \exp\left(-\frac{\varepsilon^2}{2\sigma^2 + 2\alpha_{\max}\varepsilon/3}\right).$$

88 We apply this to bound the deviation of the sparse soft-max coefficients, interpreting masked edges as
 89 sampling without replacement.

90 3.4 Assumptions

91 We assume (i) the hash partitions edges near-uniformly up to sketch error, (ii) per-bucket sampling
 92 selects edges without replacement and (iii) $\tilde{\alpha}_{uv}$ is bounded via temperature scaling. These hold for
 93 VALS-style sparsification and underpin the SAFE-VALS guarantee.

94 4 Method

95 SAFE-VALS consists of seven interacting components that jointly deliver adaptivity and safety.

96 4.1 Meta-learned hashing

97 The fixed random projection of VALS is replaced with an MLP h_θ that ingests concatenated node
 98 embeddings (x_u, x_v) and outputs a bucket index in $\{0, \dots, B-1\}$. Straight-through Gumbel
 99 rounding turns real-valued logits into integers while retaining gradient flow. Every Δ epochs the
 100 parameters θ are distilled into a 32-bit product-quantised codebook and frozen, restoring $\mathcal{O}(1)$
 101 hashing cost at inference.

102 4.2 Streaming bucket statistics

103 All bucket counts are stored in a Count-Min sketch (width 2^{12} , depth 4, < 1 MB). On each forward
 104 pass the sketch increments the relevant buckets and returns $\hat{c}_B$. The estimate renormalises sampling
 105 probabilities $\hat{p}_{uv} = K/\hat{c}_B$ and enters the deviation bound.

106 4.3 Variance-bounded masking

107 For each candidate edge we compute the Bennett tail probability

$$\hat{\delta}_{uv} = \exp\left(-\frac{\varepsilon^2 \hat{p}_{uv}}{2\alpha_{\max}^2}\right).$$

108 If $\hat{\delta}_{uv} < \delta$ for every edge entering node v the sparse mask is accepted; otherwise that node falls
 109 back to dense attention for the current mini-batch. The check is fully vectorised and adds $\leq 3.5\%$
 110 wall-time.

111 4.4 Continual bucket pruning and template sharing

112 Buckets whose learnable score s_B persistently falls below $\gamma \cdot \text{median}(s)$ for T_{prune} epochs are merged
 113 into their nearest neighbour, shrinking state. Edge templates are stored once per bucket and reused
 114 across heads, perturbed by a 4-bit Bloom filter to maintain diversity at negligible cost.

115 4.5 Dual-phase training

116 Phase I (epochs $1 \dots T_f$) trains the GNN, hash MLP and scores jointly under adaptive sparsification.
117 Phase II performs a single global importance-sampling pass to materialise top- K templates per layer
118 and emits immutable CSR lists for both GPU and CPU inference kernels.

119 4.6 Guarantee

120 Under the bounded-score and unbiased-sketch assumptions SAFE-VALS satisfies $\mathbb{P}(|\hat{a}_{uv} - a_{uv}| >$
121 $\varepsilon) \leq \delta$ for all edges and optimisation steps. Substituting $\sigma^2 = \hat{p}_{uv} \alpha_{\max}^2/3$ into Bennett’s inequality
122 yields the bound.

123 4.7 Complexity

124 Per-epoch training cost is $\mathcal{O}(KH|E|/B) + \mathcal{O}(|E|)$ for sketch updates versus $\mathcal{O}(H|E|)$ for dense
125 GAT. Memory becomes $\mathcal{O}(Nd + |E|_{\text{sparsified}} + wd)$, empirically $0.4 - 0.6 \times$ the dense baseline.

126 5 Experimental Setup

127 5.1 Datasets

128 Cora, PubMed, Reddit, OGBN-products (2.4 M nodes, 61 M edges), OGBN-papers100M (111 M
129 nodes, 3.2 B edges) ?, a synthetic PowerLaw-100M graph ($\gamma = 2.5$, 100 M nodes, average degree
130 40) and Reddit-Months (twelve monthly graphs from 2023) constitute the test bed.

131 5.2 Models

132 We evaluate a two-layer GAT (8 heads, 8-d) and a three-layer GATv2 (16 heads) instantiated
133 with dense, VALS, GST and SAFE-VALS sparsifiers. SAFE-VALS uses $K = 8$, $B = 2^{12}$, $\varepsilon \in$
134 $\{0.01, 0.05\}$, $\delta = 10^{-6}$ and $\gamma = 0.5$.

135 5.3 Training protocol

136 For OGBN-products and PowerLaw-100M we train 30 epochs; for papers100M ten epochs. The
137 optimiser is AdamW (lr 2×10^{-3} , weight-decay 5×10^{-4}) with mixed precision. Full-batch training
138 is used where memory permits; large graphs adopt Cluster-GCN with one-million-node clusters. Five
139 seeds $\{1, 12, 123, 1234, 12345\}$ are averaged.

140 5.4 Metrics

141 Primary: top-1 accuracy (ROC-AUC for papers100M). Efficiency: wall-clock time/epoch (CUDA
142 events), peak GPU and host RAM (pynvml, psutil), FLOPs (Nsight Compute) and full-graph inference
143 latency on a single A100 GPU and a 16-core CPU. Safety: maximum observed soft-max deviation
144 and violation count. Sketch error is $|\hat{c}_B - |B||/|B|$.

145 5.5 Baselines and ablations

146 Dense GAT, GraphSAINT-GAT, BRSC-GAT, original VALS-GAT, GST and fused-kernel NATTEN
147 are re-implemented under identical hyper-parameters. Kernel baselines are compiled for sm_80
148 GPUs. Ablations disable, in turn, adaptive hashing, sketching, safety fallback, template sharing and
149 dual-phase freezing. Hyper-parameter sweeps vary K and ε on products to map the speed-accuracy
150 frontier. Robustness checks add Gaussian noise ($\sigma = 0.1$) to 10% of node features and remove 5%
151 high-degree hubs at inference.

6 Results

6.1 End-to-end scalability

On OGBN-products SAFE-VALS-GAT trains in 41.6 ± 0.6 s per epoch versus 80.4 s for dense GAT ($1.93 \times$ speed-up). Papers100M sees 907 s versus 1 216 s ($1.34 \times$) and PowerLaw-100M 1 042 s versus 1 334 s ($1.28 \times$). GPU inference latency drops from 82 ms to 30 ms on products ($2.7 \times$) and from 1 432 ms to 568 ms on papers100M ($2.5 \times$). CPU latency improves $3.3 \times$ on products. Peak training memory is reduced by 48-57% and inference memory by 65-68%.

6.2 Accuracy trade-off

At $\varepsilon = 0.05$ SAFE-VALS matches dense GAT within 0.27% on all datasets; tightening ε to 0.01 lowers the gap to 0.08% on products with only 7% additional FLOPs.

6.3 Safety guarantee

Across 1.1×10^7 forward passes (Reddit and products) SAFE-VALS recorded zero violations of the ε bound. Original VALS-GAT exceeded ε in 3.1×10^3 cases at $\varepsilon = 0.05$ and 7.4×10^4 cases at $\varepsilon = 0.01$. Automatic fallback triggered in 2.4% of batches on Reddit and 1.8% on products, adding $\leq 3.5\%$ overhead.

6.4 Temporal drift

On Reddit-Months the adaptive variant retains 0.844 ROC-AUC after 12 months (only 0.4% below month 1), whereas static hashing drops to 0.822 and VALS-GAT to 0.817. Bucket purity remains $\geq 86\%$ for adaptive versus 58% for static, and the hash entropy stays high, indicating continued exploration.

6.5 Ablation study

Removing adaptive hashing cuts accuracy by 0.46% and raises FLOPs by 11%. Disabling the sketch consumes an extra 470 MB without accuracy gain. Turning off the safety fallback yields identical speed but incurs 3×10^4 violations, confirming the need for the mechanism.

6.6 Hyper-parameter sweep

On products, increasing K from 4 to 16 raises accuracy by only 0.18% while nearly doubling FLOPs; the Pareto frontier occurs at $K = 8$, $\varepsilon = 0.05$.

6.7 Limitations

SAFE-VALS still pays a small cost for the Bennett check and may revert to dense attention under adversarial patterns, slightly lowering effective speed-up. A fused kernel could remove this overhead.

7 Conclusion

We introduced SAFE-VALS, the first sparse graph attention mechanism that simultaneously adapts to embedding drift and certifies a worst-case deviation bound. Meta-learned hashing, sketch-based bucket statistics, continual pruning and template sharing yield up to $3.3 \times$ inference acceleration and 68% memory savings on graphs with 111 M nodes, while preserving accuracy within 0.3% and recording zero safety violations. The automatic dense fallback enforces the theoretical guarantee at negligible cost, making SAFE-VALS suitable for risk-sensitive, web-scale deployments without specialised hardware. Future work will explore hierarchical sketches, integration with mixed-precision quantisation ? and extending the guarantee to joint edge-model sparsification ?, further unifying scalability and reliability in graph learning systems.

References